

Sí. Con la estructura que describes, **yo no resolvería esto con triggers para todo**. La arquitectura más limpia en Oracle sería:

Oracle Database → procedimientos/package PL/SQL → DBMS_SCHEDULER → UTL_SMTP → Oracle Forms Builder

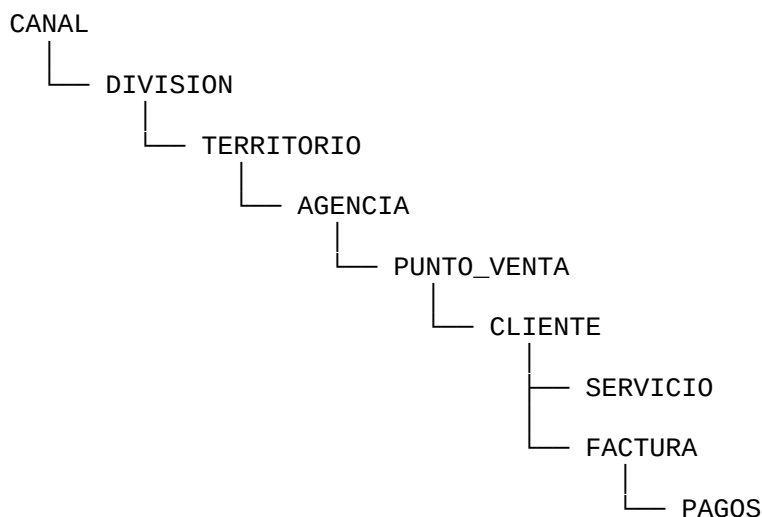
Y utilizaría **un proceso central que determine si un punto de venta debe estar bloqueado o desbloqueado**, en lugar de tener lógica repetida en varios lugares.

Además, hay una distinción importante: `DBMS_SCHEDULER.ADD_JOB_EMAIL_NOTIFICATION` sirve principalmente para enviar notificaciones sobre **eventos del propio job** —por ejemplo, que falló o terminó—, no para generar tu reporte de mora con un Excel adjunto. Oracle documenta `ADD_JOB_EMAIL_NOTIFICATION` precisamente como mecanismo de notificación de eventos del Scheduler.

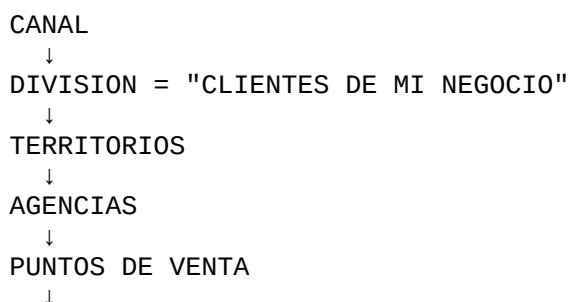
Para tu caso, usaría `DBMS_SCHEDULER` para ejecutar el proceso y `UTL_SMTP` para construir el correo y adjuntar el archivo.

1. Arquitectura que recomiendo

La relación que entiendo es:



Y la regla principal es:



CLIENTES
↓
FACTURAS/SERVICIOS
↓
PAGOS

Solamente se consideran los puntos de venta que pertenezcan a la división CLIENTES DE MI NEGOCIO.

La regla de bloqueo sería:

Cliente tiene factura vencida
↓
Existe saldo pendiente
↓
No existe arreglo de pago vigente
↓
Cliente está en mora
↓
Punto de venta pasa a INACTIVO

Pero hay algo que recomiendo agregar:

Punto de venta
estado = ACTIVO / INACTIVO

motivo_estado =
 NORMAL
 MORA
 MANUAL
 OTRO

origen_estado =
 AUTOMATICO
 MANUAL

Esto es importante porque si un usuario bloquea manualmente un punto de venta, el job automático no debería desbloquearlo simplemente porque el cliente no está en mora.

2. Antes de programar: modelo de datos

Como todavía no me diste los nombres reales de todas las columnas, voy a utilizar nombres ficticios coherentes.

Después podemos adaptar exactamente el código a tus tablas reales.

Supongamos:

CANAL

CANAL

ID_CANAL

NOMBRE
ESTADO

DIVISION

DIVISION

ID_DIVISION
ID_CANAL
NOMBRE
ESTADO

TERRITORIO

TERRITORIO

ID_TERRITORIO
ID_DIVISION
NOMBRE
ESTADO

AGENCIA

AGENCIA

ID_AGENCIA
ID_TERRITORIO
NOMBRE
ESTADO

PUNTO_VENTA

PUNTO_VENTA

ID_PUNTO_VENTA
ID_AGENCIA
NOMBRE
ESTADO

CLIENTE

CLIENTE

ID_CLIENTE
ID_PUNTO_VENTA
NOMBRE
ESTADO

SERVICIO

SERVICIO

ID_SERVICIO
ID_CLIENTE

PUNTO_VENTA
DESCRIPCION
MONTO_ANUAL
FECHA_INICIO
FECHA_FIN
ESTADO

Aquí tomo literalmente tu indicación:

en SERVICIOS es donde se encuentra PUNTO_VENTA.

FACTURA

FACTURA

ID_FACTURA
ID_SERVICIO
ID_CLIENTE
FECHA_EMISION
FECHA_VENCIMIENTO
MONTO
SALDO
ESTADO

PAGO

PAGO

ID_PAGO
ID_FACTURA
FECHA_PAGO
MONTO

ARREGLO_PAGO

ARREGLO_PAGO

ID_ARREGLO
ID_CLIENTE
ID_FACTURA
FECHA_INICIO
FECHA_FIN
ESTADO

Por ejemplo:

ACTIVO
CUMPLIDO
INCUMPLIDO
CANCELADO

3. Lo primero que haría: crear una vista de mora

No pondría toda la lógica de mora directamente dentro del Scheduler.

Crearía una vista.

Por ejemplo:

```
CREATE OR REPLACE VIEW VW_CLIENTES_MORA AS
SELECT
    c.id_cliente,
    c.nombre AS cliente,
    pv.id_punto_venta,
    pv.nombre AS punto_venta,
    f.id_factura,
    f.fecha_vencimiento,
    f.monto,
    f.saldo,

    TRUNC(SYSDATE - f.fecha_vencimiento) AS dias_mora,

    d.id_division,
    d.nombre AS division
FROM cliente c
JOIN punto_venta pv
    ON pv.id_punto_venta = c.id_punto_venta
JOIN agencia a
    ON a.id_agencia = pv.id_agencia
JOIN territorio t
    ON t.id_territorio = a.id_territorio
JOIN division d
    ON d.id_division = t.id_division
JOIN factura f
    ON f.id_cliente = c.id_cliente
WHERE UPPER(TRIM(d.nombre)) = 'CLIENTES DE MI NEGOCIO'
AND f.fecha_vencimiento < TRUNC(SYSDATE)
AND NVL(f.saldo, 0) > 0
AND NOT EXISTS (
    SELECT 1
    FROM arreglo_pago ap
    WHERE ap.id_cliente = c.id_cliente
        AND ap.id_factura = f.id_factura
        AND ap.estado = 'ACTIVO'
);
```

Esto te da algo parecido a:

Cliente	Punto venta	Factura	Vencimiento	Saldo	Días mora
Juan Pérez	PV-001	F-1001	10/09/2026	80	10
María López	PV-002	F-1008	01/09/2026	120	19
Pedro Gómez	PV-003	F-1015	15/08/2026	250	36

4. Regla importante: un punto de venta puede tener varios clientes

Aquí hay que decidir algo fundamental.

Supongamos:

PV-001
├─ Cliente A → al día
├─ Cliente B → al día
└─ Cliente C → 20 días mora

¿Debe bloquearse PV-001?

Por lo que describes, **sí**.

Por tanto:

SI EXISTE AL MENOS UN CLIENTE EN MORA
↓
PUNTO DE VENTA = INACTIVO

Y cuando **ningún cliente** tenga mora:

TODOS LOS CLIENTES AL DÍA
↓
PUNTO DE VENTA = ACTIVO

con la excepción de bloqueos manuales.

5. Tabla de historial

Esta tabla para mí es obligatoria.

```
CREATE TABLE HISTORIAL_ESTADO_PV (  
  ID_HISTORIAL          NUMBER GENERATED BY DEFAULT AS IDENTITY,  
  ID_PUNTO_VENTA        NUMBER NOT NULL,  
  ESTADO_ANTERIOR       VARCHAR2(20),  
  ESTADO_NUEVO          VARCHAR2(20),  
  
  MOTIVO                VARCHAR2(100),
```

```

        TIPO_ACCION          VARCHAR2(30),

        ID_CLIENTE           NUMBER,
        ID_FACTURA           NUMBER,

        DIAS_MORA             NUMBER,
        MONTO_MORA            NUMBER(15,2),

        USUARIO               VARCHAR2(100),
        FECHA_ACCION          DATE DEFAULT SYSDATE,

        OBSERVACION           VARCHAR2(500),

        CONSTRAINT PK_HISTORIAL_ESTADO_PV
        PRIMARY KEY (ID_HISTORIAL)
);

```

Ejemplo:

ID	PV	ANTERIOR	NUEVO	TIPO	CLIENTE	DIAS	MONTO	USUARIO
1	1001	ACTIVO	INACTIVO	AUTOMATICO	50001	15	120	JOB_MORA
2	1002	ACTIVO	INACTIVO	AUTOMATICO	50015	32	450	JOB_MORA
3	1001	INACTIVO	ACTIVO	AUTOMATICO	50001	0	0	JOB_MORA
4	1003	ACTIVO	INACTIVO	MANUAL	NULL	0	0	FBA

Esto te permite responder posteriormente:

¿Quién bloqueó este punto de venta?

¿Cuándo?

¿Por qué?

¿Fue automático o manual?

¿Qué factura provocó el bloqueo?

6. Agregaría dos campos a PUNTO_VENTA

Si tienes autorización para modificar la tabla:

```

ALTER TABLE PUNTO_VENTA ADD (
    MOTIVO_ESTADO VARCHAR2(30),
    ORIGEN_ESTADO VARCHAR2(20)
);

```

Valores:

MOTIVO_ESTADO

NORMAL
MORA

MANUAL

y:

ORIGEN_ESTADO

AUTOMATICO

MANUAL

Ejemplo:

PV-001

ESTADO = INACTIVO

MOTIVO_ESTADO = MORA

ORIGEN_ESTADO = AUTOMATICO

Esto simplifica muchísimo la administración.

7. No recomiendo una FUNCTION para esto

Para cambiar datos:

FUNCTION no sería mi primera opción.

Utilizaría:

PACKAGE

↓

PROCEDURES

Por ejemplo:

PKG_CONTROL_MORA

- PROCESAR_MORA
- BLOQUEAR_PUNTO_VENTA
- DESBLOQUEAR_PUNTO_VENTA
- BLOQUEAR_MANUAL
- DESBLOQUEAR_MANUAL
- GENERAR_REPORTE_MORA

Esto centraliza toda la lógica.

8. Procedimiento para bloquear/desbloquear

Primero:

```
CREATE OR REPLACE PROCEDURE SP_BLOQUEAR_PV_MORA (  
    P_ID_PUNTO_VENTA IN NUMBER,  
    P_ID_CLIENTE      IN NUMBER,
```

```

P_ID_FACTURA      IN NUMBER,
P_DIAS_MORA       IN NUMBER,
P_MONTO_MORA      IN NUMBER
)
IS
V_ESTADO_ACTUAL PUNTO_VENTA.ESTADO%TYPE;
BEGIN

SELECT ESTADO
INTO V_ESTADO_ACTUAL
FROM PUNTO_VENTA
WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA
FOR UPDATE;

IF V_ESTADO_ACTUAL <> 'INACTIVO' THEN

    UPDATE PUNTO_VENTA
    SET ESTADO = 'INACTIVO',
        MOTIVO_ESTADO = 'MORA',
        ORIGEN_ESTADO = 'AUTOMATICO'
    WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA;

    INSERT INTO HISTORIAL_ESTADO_PV (
        ID_PUNTO_VENTA,
        ESTADO_ANTERIOR,
        ESTADO_NUEVO,
        MOTIVO,
        TIPO_ACCION,
        ID_CLIENTE,
        ID_FACTURA,
        DIAS_MORA,
        MONTO_MORA,
        USUARIO,
        FECHA_ACCION,
        OBSERVACION
    )
    VALUES (
        P_ID_PUNTO_VENTA,
        V_ESTADO_ACTUAL,
        'INACTIVO',
        'MORA',
        'BLOQUEO_AUTOMATICO',
        P_ID_CLIENTE,
        P_ID_FACTURA,
        P_DIAS_MORA,
        P_MONTO_MORA,
        'JOB_MORA',
        SYSDATE,
        'Bloqueo automático por mora'
    );

END IF;

END;
/

```

9. Proceso central de bloqueo

Ahora hacemos el procedimiento importante.

```
CREATE OR REPLACE PROCEDURE SP_PROCESAR_BLOQUEOS_MORA
IS
BEGIN
    FOR R IN (
        SELECT
            id_punto_venta,
            id_cliente,
            id_factura,
            dias_mora,
            saldo
        FROM VW_CLIENTES_MORA
    )
    LOOP
        SP_BLOQUEAR_PV_MORA(
            P_ID_PUNTO_VENTA => R.id_punto_venta,
            P_ID_CLIENTE      => R.id_cliente,
            P_ID_FACTURA      => R.id_factura,
            P_DIAS_MORA       => R.dias_mora,
            P_MONTO_MORA      => R.saldo
        );
    END LOOP;

    COMMIT;

EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
        RAISE;
END;
/
```

10. ¿Cada cuánto ejecutarlo?

Aquí recomiendo:

Cada 3 horas.

Por ejemplo:

```
06:00
09:00
12:00
15:00
18:00
21:00
```

Oracle Scheduler permite definir estas recurrencias mediante `repeat_interval`. Oracle documenta `FREQ`, `INTERVAL`, `BYHOUR`, `BYMINUTE`, etc.

El job:

BEGIN

```
DBMS_SCHEDULER.CREATE_JOB (  
    job_name          => 'JOB_CONTROL_MORA',  
    job_type          => 'STORED_PROCEDURE',  
    job_action        => 'SP_PROCESAR_BLOQUEOS_MORA',  
    start_date        => TO_TIMESTAMP_TZ(  
                        '20-SEP-2026 06:00:00 AM America/Managua',  
                        'DD-MON-YYYY HH:MI:SS AM TZR'  
                    ),  
    repeat_interval   =>  
        'FREQ=HOURLY;INTERVAL=3;BYMINUTE=0;BYSECOND=0',  
    enabled           => FALSE,  
    auto_drop         => FALSE,  
    comments          => 'Bloqueo automático de puntos de venta por mora'  
);  
  
DBMS_SCHEDULER.ENABLE('JOB_CONTROL_MORA');
```

END;
/

Sin embargo, si quieres exactamente:

```
06:00  
09:00  
12:00  
15:00  
...
```

puedes hacerlo de manera explícita:

```
repeat_interval =>  
    'FREQ=DAILY;BYHOUR=6,9,12,15,18,21;BYMINUTE=0;BYSECOND=0'
```

Yo prefiero esta última porque expresa exactamente el negocio.

11. Desbloqueo automático

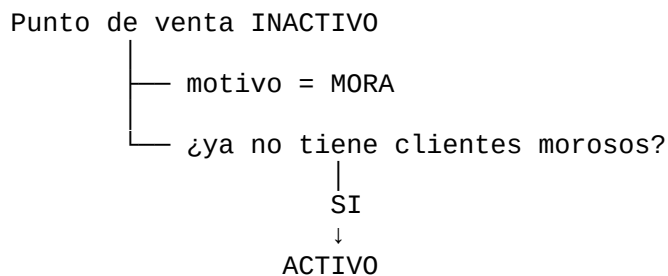
Este punto es muy importante.

No debes hacer simplemente:

```
UPDATE PUNTO_VENTA  
SET ESTADO = 'ACTIVO';
```

porque podrías desbloquear un punto de venta bloqueado manualmente.

La lógica debe ser:



Por eso:

```
CREATE OR REPLACE PROCEDURE SP_PROCESAR_DESBLOQUEOS
IS
BEGIN
    FOR R IN (
        SELECT pv.id_punto_venta
        FROM punto_venta pv
        WHERE pv.estado = 'INACTIVO'
          AND pv.motivo_estado = 'MORA'

        AND NOT EXISTS (
            SELECT 1
            FROM VW_CLIENTES_MORA vm
            WHERE vm.id_punto_venta = pv.id_punto_venta
        )
    )
    LOOP

        UPDATE PUNTO_VENTA
        SET ESTADO = 'ACTIVO',
            MOTIVO_ESTADO = 'NORMAL',
            ORIGEN_ESTADO = 'AUTOMATICO'
        WHERE ID_PUNTO_VENTA = R.ID_PUNTO_VENTA;

        INSERT INTO HISTORIAL_ESTADO_PV (
            ID_PUNTO_VENTA,
            ESTADO_ANTERIOR,
            ESTADO_NUEVO,
            MOTIVO,
            TIPO_ACCION,
            USUARIO,
            FECHA_ACCION,
            OBSERVACION
        )
        VALUES (
            R.ID_PUNTO_VENTA,
            'INACTIVO',
            'ACTIVO',
            'MORA_SALDADA',
            'DESBLOQUEO_AUTOMATICO',
            'JOB_MORA',
            SYSDATE,
            'Punto de venta desbloqueado automáticamente'
        );
    END LOOP;
```

```
COMMIT;  
  
END;  
/
```

12. ¿Qué pasa con un arreglo de pago?

Aquí propondría esta regla:

```
Cliente tiene mora  
↓  
Se crea arreglo de pago  
↓  
ARREGLO_PAGO.ESTADO = ACTIVO  
↓  
Cliente deja de considerarse moroso  
↓  
PV puede desbloquearse
```

Por eso nuestra vista ya tiene:

```
AND NOT EXISTS (  
    SELECT 1  
    FROM arreglo_pago ap  
    WHERE ap.id_cliente = c.id_cliente  
          AND ap.id_factura = f.id_factura  
          AND ap.estado = 'ACTIVO'  
)
```

13. ¿Trigger para arreglo de pago?

Aquí sí puedes utilizar un trigger, pero **solo para disparar una operación sencilla**.

Por ejemplo:

```
CREATE OR REPLACE TRIGGER TRG_ARREGLO_PAGO  
AFTER INSERT OR UPDATE OF ESTADO  
ON ARREGLO_PAGO  
FOR EACH ROW  
WHEN (NEW.ESTADO = 'ACTIVO')  
BEGIN  
  
    SP_PROCESAR_DESBLOQUEOS;  
  
END;  
/
```

Pero personalmente **no lo usaría como mecanismo principal**.

Prefiero:

ARREGLO DE PAGO
↓
COMMIT
↓
JOB cada 3 horas
↓
evalúa nuevamente
↓
DESBLOQUEA

Es mucho más fácil de mantener.

Si el negocio exige desbloqueo inmediato, entonces podemos implementar el trigger.

14. Proceso para determinar la mora

Yo centralizaría además una consulta como:

```
SELECT
    id_punto_venta,
    COUNT(DISTINCT id_cliente) clientes_mora,
    COUNT(DISTINCT id_factura) facturas_mora,
    SUM(saldo) total_mora,
    MAX(dias_mora) mayor_dias_mora
FROM VW_CLIENTES_MORA
GROUP BY id_punto_venta;
```

Resultado:

	PV	Clientes	Facturas	Total mora	Mayor mora
1001	2	3	450	45	
1002	1	1	120	20	
1003	3	5	890	72	

Esto es exactamente lo que posteriormente utilizaremos para el reporte.

15. Correo automático

Aquí hay una diferencia importante.

No recomiendo:

```
DBMS_SCHEDULER
↓
ADD_JOB_EMAIL_NOTIFICATION
```

para enviar el reporte.

ADD_JOB_EMAIL_NOTIFICATION está pensado para eventos del Scheduler como:

JOB_STARTED
JOB_SUCCEEDED
JOB_FAILED
JOB_BROKEN
JOB_DISABLED

y Oracle muestra ese uso en su documentación.

Para tu correo de negocio utilizaría:

DBMS_SCHEDULER
↓
SP_ENVIAR_REPORTE_MORA
↓
UTL_SMTP
↓
SMTP corporativo
↓
3 destinatarios

Por ejemplo:

cobranzas@empresa.demo
supervision@empresa.demo
gerencia@empresa.demo

16. Configurar ACL para correo

En Oracle moderno, el esquema necesita permisos de red para conectarse al SMTP.

Ejemplo conceptual:

```
BEGIN

    DBMS_NETWORK_ACL_ADMIN.APPEND_HOST_ACE(
        host => 'smtp.empresa.local',
        lower_port => 25,
        upper_port => 25,
        ace => XS$ACE_TYPE(
            privilege_list => XS$NAME_LIST('connect'),
            principal_name => 'USUARIO_APP',
            principal_type => XS_ACL.PTYPE_DB
        )
    );

END;
/
```

Esto debe hacerlo normalmente un DBA con los privilegios correspondientes.

17. Configuración SMTP

Supongamos:

SMTP:
smtp.empresa.local

Puerto:
25

Remitente:
notificaciones@empresa.demo

Podrías crear una tabla de configuración:

```
CREATE TABLE CONFIG_CORREO (  
    ID_CONFIG NUMBER PRIMARY KEY,  
    SMTP_HOST VARCHAR2(200),  
    SMTP_PORT NUMBER,  
    EMAIL_REMITENTE VARCHAR2(200),  
    NOMBRE_REMITENTE VARCHAR2(200)  
);
```

Y:

```
INSERT INTO CONFIG_CORREO VALUES (  
    1,  
    'smtp.empresa.local',  
    25,  
    'notificaciones@empresa.demo',  
    'Sistema de Control de Mora'  
);  
  
COMMIT;
```

18. ¿Cómo crear el Excel?

Aquí hay otro punto importante.

Un archivo .xlsx real no es simplemente una tabla de texto. Es un paquete ZIP que contiene XML.

Por eso **no recomiendo intentar construir un XLSX manualmente con PL/SQL puro**, salvo que en tu empresa ya exista una librería para ello.

Tienes tres alternativas:

Opción A — CSV

Oracle genera:

puntos_venta_mora.csv

Excel lo abre perfectamente.

Es la opción más sencilla.

Opción B — HTML como .xls

Puedes generar:

reporte_mora.xls

con una tabla HTML.

Excel normalmente lo abre.

Opción C — XLSX real

Utilizar una herramienta adicional para generar XLSX.

Para tu escenario Oracle + Windows, yo empezaría con **CSV o XLS compatible**, y si el requisito corporativo exige específicamente .xlsx, implementaría un generador externo.

19. Mi recomendación para tu empresa

Si el requerimiento dice literalmente:

"Adjuntar un Excel"

yo utilizaría:

reporte_puntos_venta_mora.csv

si aceptan CSV.

Si dicen:

"Tiene que ser .xlsx"

entonces haría:

Oracle Scheduler

↓

Procedimiento PL/SQL

↓

archivo CSV temporal

↓

PowerShell/Python en Windows

↓

XLSX

↓

correo

Pero eso introduce otro componente.

20. Alternativa 100% Oracle

Puedes generar un archivo CSV desde PL/SQL utilizando UTL_FILE.

Primero:

```
CREATE OR REPLACE DIRECTORY DIR_REPORTES
AS 'C:\REPORTES_MORA';
```

Y el DBA debe otorgar:

```
GRANT READ, WRITE ON DIRECTORY DIR_REPORTES TO USUARIO_APP;
```

Entonces:

```
CREATE OR REPLACE PROCEDURE SP_GENERAR_CSV_MORA
IS
    V_FILE UTL_FILE.FILE_TYPE;
BEGIN
    V_FILE := UTL_FILE.FOPEN(
        'DIR_REPORTES',
        'reporte_mora.csv',
        'W',
        32767
    );

    UTL_FILE.PUT_LINE(
        V_FILE,
        'Punto Venta,Cliente,Factura,Dias Mora,Saldo'
    );

    FOR R IN (
        SELECT
            id_punto_venta,
            cliente,
            id_factura,
            dias_mora,
            saldo
        FROM VW_CLIENTES_MORA
        ORDER BY id_punto_venta
    )
    LOOP

        UTL_FILE.PUT_LINE(
            V_FILE,
            R.ID_PUNTO_VENTA || ',' ||
            '"' || REPLACE(R.CLIENTE, '"', '""') || ',' ||
            R.ID_FACTURA || ',' ||
            R.DIAS_MORA || ',' ||
            R.SALDO
        );
    END LOOP;
END;
```

```
END LOOP;  
  
UTL_FILE.FCLOSE(V_FILE);  
  
END;  
/
```

Archivo:

C:\REPORTES_MORA\reporte_mora.csv

Excel puede abrirlo.

21. Ejemplo del reporte

```
Punto Venta, Cliente, Factura, Dias Mora, Saldo  
1001, Juan Perez, 50001, 15, 120.00  
1001, Maria Lopez, 50008, 25, 300.00  
1002, Pedro Gomez, 50011, 7, 80.00  
1003, Ana Garcia, 50019, 45, 550.00
```

22. Correo con reporte

El procedimiento puede generar un HTML como cuerpo:

ASUNTO:

Reporte automático de mora - 20/09/2026 06:00 AM

Contenido:

Estimados,

Se presenta el reporte de puntos de venta afectados por mora.

```
Puntos de venta bloqueados: 12  
Clientes en mora: 18  
Total pendiente: $4,580.00
```

El detalle se encuentra en el archivo adjunto.

Sistema de Control de Mora

Y adjuntar:

reporte_mora_20260920_0600.csv

23. Job de reporte cada 3 horas

Podemos crear un segundo job:

BEGIN

```
DBMS_SCHEDULER.CREATE_JOB (  
  job_name          => 'JOB_REPORTER_MORA',  
  job_type          => 'STORED_PROCEDURE',  
  job_action        => 'SP_ENVIAR_REPORTER_MORA',  
  start_date        => TO_TIMESTAMP_TZ(  
    '20-SEP-2026 06:00:00 AM America/Managua',  
    'DD-MON-YYYY HH:MI:SS AM TZR'  
  ),  
  repeat_interval   =>  
    'FREQ=DAILY;BYHOUR=6,9,12,15,18,21;BYMINUTE=0;BYSECOND=0',  
  enabled           => TRUE,  
  auto_drop         => FALSE,  
  comments          => 'Reporte de puntos de venta en mora'  
);
```

END;
/

Oracle Scheduler soporta este tipo de calendarios recurrentes mediante `repeat_interval`.

24. Pero yo haría un solo Job

En realidad, después de analizar todo tu requerimiento, **cambiaría ligeramente la arquitectura**.

En lugar de:

JOB 1 → bloqueo
JOB 2 → desbloqueo
JOB 3 → correo

utilizaría:

```
JOB_CONTROL_MORA  
├── 1. detectar mora  
├── 2. bloquear PV  
├── 3. detectar arreglos de pago  
├── 4. desbloquear PV  
├── 5. registrar historial  
└── 6. generar reporte  
    └── correo
```

Un solo proceso cada 3 horas.

Esto reduce posibilidades de que:

JOB BLOQUEO

se ejecute a las 09:00 y:

JOB DESBLOQUEO

se ejecute después, con información diferente.

25. Package que finalmente recomiendo

En producción haría:

```
CREATE OR REPLACE PACKAGE PKG_CONTROL_MORA AS
```

```
    PROCEDURE PROCESAR_MORA;
```

```
    PROCEDURE BLOQUEAR_PV(  
        P_ID_PUNTO_VENTA NUMBER,  
        P_ID_CLIENTE NUMBER,  
        P_ID_FACTURA NUMBER,  
        P_DIAS_MORA NUMBER,  
        P_MONTO NUMBER  
    );
```

```
    PROCEDURE DESBLOQUEAR_PV(  
        P_ID_PUNTO_VENTA NUMBER  
    );
```

```
    PROCEDURE BLOQUEAR_MANUAL(  
        P_ID_PUNTO_VENTA NUMBER,  
        P_USUARIO VARCHAR2,  
        P_MOTIVO VARCHAR2  
    );
```

```
    PROCEDURE DESBLOQUEAR_MANUAL(  
        P_ID_PUNTO_VENTA NUMBER,  
        P_USUARIO VARCHAR2,  
        P_MOTIVO VARCHAR2  
    );
```

```
    PROCEDURE GENERAR_REPORTE;
```

```
    PROCEDURE ENVIAR_REPORTE;
```

```
END PKG_CONTROL_MORA;  
/
```

Esto te deja una arquitectura mucho más profesional.

26. Bloqueo manual desde Oracle Forms

En Forms Builder crearía una opción de menú:

Administración

- Usuarios
- Parámetros
- Control de Puntos de Venta

Dentro:

CONTROL DE PUNTOS DE VENTA

Formulario:

CONTROL DE PUNTO DE VENTA	
Punto Venta: [1001] [Buscar]	
Nombre:	PV Managua Centro
Agencia:	Agencia Central
Territorio:	Managua
División:	Clientes de mi negocio
Estado actual:	INACTIVO
Motivo:	MORA
Cliente moroso:	Juan Pérez
Días mora:	25
Total mora:	\$450.00
Motivo acción: [_____]	
[BLOQUEAR]	[DESBLOQUEAR]

27. Forms no debería hacer directamente el UPDATE

Esto es importante.

No haría:

```
UPDATE PUNTO_VENTA  
SET ESTADO = 'INACTIVO';
```

directamente desde Forms.

Forms debería llamar al package:

```
PKG_CONTROL_MORA.BLOQUEAR_MANUAL(  
    :PUNTO_VENTA.ID_PUNTO_VENTA,  
    USER,  
    :CONTROL.MOTIVO  
);
```

Así:

```
Oracle Forms  
  ↓  
PKG_CONTROL_MORA  
  ↓  
PUNTO_VENTA  
  +  
HISTORIAL_ESTADO_PV
```

La lógica queda en la base de datos.

28. Procedimiento manual

```
CREATE OR REPLACE PROCEDURE SP_BLOQUEAR_PV_MANUAL (  
    P_ID_PUNTO_VENTA IN NUMBER,  
    P_USUARIO          IN VARCHAR2,  
    P_MOTIVO           IN VARCHAR2  
)  
IS  
    V_ESTADO PUNTO_VENTA.ESTADO%TYPE;  
BEGIN  
  
    SELECT ESTADO  
    INTO V_ESTADO  
    FROM PUNTO_VENTA  
    WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA  
    FOR UPDATE;  
  
    UPDATE PUNTO_VENTA  
    SET ESTADO = 'INACTIVO',  
        MOTIVO_ESTADO = 'MANUAL',  
        ORIGEN_ESTADO = 'MANUAL'  
    WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA;  
  
    INSERT INTO HISTORIAL_ESTADO_PV (  
        ID_PUNTO_VENTA,  
        ESTADO_ANTERIOR,  
        ESTADO_NUEVO,  
        MOTIVO,  
        TIPO_ACCION,  
        USUARIO,  
        FECHA_ACCION,  
        OBSERVACION  
    )  
    VALUES (  
        P_ID_PUNTO_VENTA,
```

```

        V_ESTADO,
        'INACTIVO',
        'MANUAL',
        'BLOQUEO_MANUAL',
        P_USUARIO,
        SYSDATE,
        P_MOTIVO
    );

    COMMIT;

END;
/

```

29. Desbloqueo manual

```

CREATE OR REPLACE PROCEDURE SP_DESBLOQUEAR_PV_MANUAL (
    P_ID_PUNTO_VENTA IN NUMBER,
    P_USUARIO          IN VARCHAR2,
    P_MOTIVO           IN VARCHAR2
)
IS
    V_ESTADO PUNTO_VENTA.ESTADO%TYPE;
BEGIN

    SELECT ESTADO
    INTO V_ESTADO
    FROM PUNTO_VENTA
    WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA
    FOR UPDATE;

    UPDATE PUNTO_VENTA
    SET ESTADO = 'ACTIVO',
        MOTIVO_ESTADO = 'NORMAL',
        ORIGEN_ESTADO = 'MANUAL'
    WHERE ID_PUNTO_VENTA = P_ID_PUNTO_VENTA;

    INSERT INTO HISTORIAL_ESTADO_PV (
        ID_PUNTO_VENTA,
        ESTADO_ANTERIOR,
        ESTADO_NUEVO,
        MOTIVO,
        TIPO_ACCION,
        USUARIO,
        FECHA_ACCION,
        OBSERVACION
    )
    VALUES (
        P_ID_PUNTO_VENTA,
        V_ESTADO,
        'ACTIVO',
        'MANUAL',
        'DESBLOQUEO_MANUAL',
        P_USUARIO,
        SYSDATE,

```

```
        P_MOTIVO
    );

    COMMIT;

END;
/
```

30. Seguridad: algo que recomiendo fuertemente

No todos los usuarios deberían poder desbloquear.

Por ejemplo:

Usuario operador
→ consultar

Supervisor
→ bloquear manualmente

Gerente
→ bloquear/desbloquear

Esto lo puedes controlar desde Forms y, mejor aún, desde una tabla de permisos.

31. ¿Cómo saber quién ejecutó el job?

Para procesos automáticos:

```
SYS_CONTEXT(
    'USERENV',
    'SESSION_USER'
)
```

te puede identificar el usuario de BD.

Pero para Forms puedes guardar:

```
:GLOBAL.USUARIO
```

o el usuario real de aplicación.

Por ejemplo:

```
USUARIO = FBALLADARES
```

mientras que:

```
TIPO_ACCION = BLOQUEO_MANUAL
```

32. Historial también debería guardar el motivo real

Ejemplo:

PV: 1001

20/09/2026 06:00
ACTIVO → INACTIVO
Motivo: MORA
Cliente: 5001
Factura: 90001
Días mora: 35
Monto: \$250
Usuario: JOB_MORA

Luego:

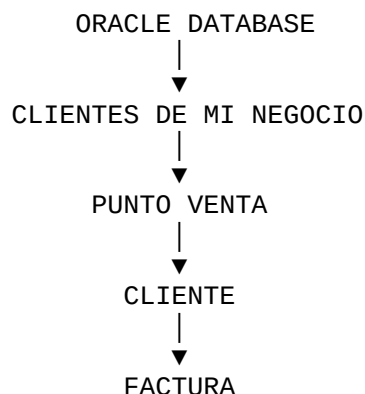
22/09/2026 09:00
INACTIVO → ACTIVO
Motivo: ARREGLO_PAGO
Cliente: 5001
Usuario: JOB_MORA

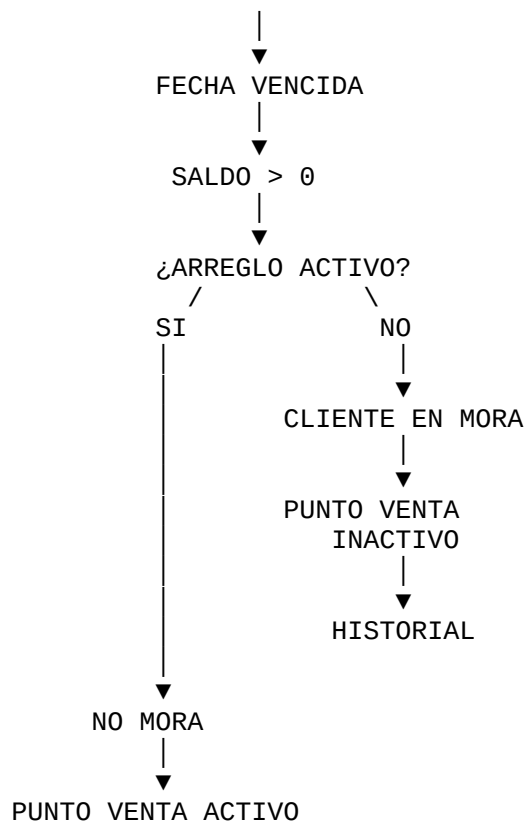
Y si un usuario manualmente lo vuelve a bloquear:

23/09/2026 14:35
ACTIVO → INACTIVO
Motivo: MANUAL
Usuario: SUPERVISOR01
Observación: Solicitud de administración

Tienes una auditoría completa.

33. El proceso completo quedaría así



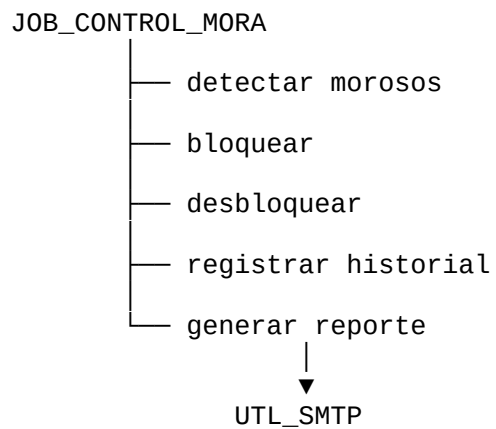


34. Scheduler

Oracle:

06:00
09:00
12:00
15:00
18:00
21:00

Cada ejecución:



|
— cobranzas@empresa.demo
— supervision@empresa.demo
— gerencia@empresa.demo

35. Una mejora importante: no bloquear por una sola factura accidental

Yo pondría una regla de negocio configurable.

Por ejemplo:

MORA > 0 días → bloqueo

o:

MORA >= 5 días → bloqueo

o:

MORA >= 1 factura vencida → bloqueo

Esto debería estar en una tabla de parámetros:

```
CREATE TABLE PARAMETROS_MORA (  
    ID_PARAMETRO NUMBER PRIMARY KEY,  
    DIAS_MIN_MORA NUMBER,  
    BLOQUEO_AUTOMATICO CHAR(1),  
    ENVIO_CORREO CHAR(1)  
);
```

Ejemplo:

```
INSERT INTO PARAMETROS_MORA  
VALUES (1, 1, 'S', 'S');
```

```
COMMIT;
```

Entonces mañana no necesitas modificar código para cambiar:

1 día

a:

3 días

36. Índices que probablemente vas a necesitar

Con este volumen de información, los índices serán importantes.

Por ejemplo:

```
CREATE INDEX IDX_FACTURA_CLIENTE  
ON FACTURA(ID_CLIENTE);
```

```
CREATE INDEX IDX_FACTURA_VENCIMIENTO  
ON FACTURA(FECHA_VENCIMIENTO);
```

```
CREATE INDEX IDX_FACTURA_ESTADO  
ON FACTURA(ESTADO);
```

```
CREATE INDEX IDX_ARREGLO_CLIENTE  
ON ARREGLO_PAGO(ID_CLIENTE);
```

```
CREATE INDEX IDX_ARREGLO_FACTURA  
ON ARREGLO_PAGO(ID_FACTURA);
```

```
CREATE INDEX IDX_CLIENTE_PV  
ON CLIENTE(ID_PUNTO_VENTA);
```

```
CREATE INDEX IDX_PV_AGENCIA  
ON PUNTO_VENTA(ID_AGENCIA);
```

Y dependiendo de tus volúmenes reales, podemos optimizar los índices después.

37. No usaría un trigger sobre FACTURA para bloquear inmediatamente

Por ejemplo, **no recomiendo**:

```
INSERT FACTURA  
  ↓  
TRIGGER  
  ↓  
consulta 10 tablas  
  ↓  
bloquea PV
```

¿Por qué?

Porque la factura puede todavía estar dentro de una transacción.

Además, la mora realmente depende de:

```
fecha actual  
+  
vencimiento  
+
```

saldo
+
pagos
+
arreglo

La mora cambia simplemente porque pasa el tiempo.

Por eso un trigger no es el mecanismo natural.

Scheduler es mucho más apropiado.

38. Tampoco usaría un trigger para cada pago

Un pago podría disparar:

UPDATE FACTURA
↓
TRIGGER
↓
desbloqueo

pero existen muchos escenarios:

pago parcial
pago total
pago reversado
pago aplicado a otra factura
arreglo
etc.

El proceso central puede reevaluar todo.

39. Arquitectura final que recomiendo

Capa 1 — Datos

CANAL
DIVISION
TERRITORIO
AGENCIA
PUNTO_VENTA
CLIENTE
SERVICIO
FACTURA
PAGO
ARREGLO_PAGO

Capa 2 — Vista

VW_CLIENTES_MORA

Capa 3 — Auditoría

HISTORIAL_ESTADO_PV

Capa 4 — Lógica

PKG_CONTROL_MORA

Capa 5 — Scheduler

JOB_CONTROL_MORA

Capa 6 — Comunicación

UTL_SMTP

UTL_FILE

Capa 7 — Interfaz

Oracle Forms Builder

40. Estimación de desarrollo

Para un desarrollador que ya conoce Oracle:

Tarea	Tiempo estimado
Análisis de tablas reales	2–4 h
Vista de clientes en mora	2–4 h
Tabla de historial	1–2 h
Package de bloqueo/desbloqueo	4–6 h
Job Scheduler	1–2 h
Arreglo de pago	2–4 h
Configuración SMTP	2–4 h
Generación CSV/Excel compatible	3–5 h
Correo con adjunto	3–5 h
Formulario Forms	6–10 h
Menú y permisos	2–4 h
Pruebas	5–8 h
Pruebas de casos especiales	3–5 h

Total aproximado

36–63 horas, dependiendo principalmente de cómo estén realmente diseñadas tus tablas y de la infraestructura SMTP.

Si ya existe configuración SMTP en la empresa y tablas bien relacionadas, podría bajar considerablemente.

41. Casos de prueba que yo prepararía

Antes de ponerlo en producción, crearía mínimo estos casos:

Caso 1

Cliente al día
PV ACTIVO

Resultado:

ACTIVO

Caso 2

Cliente
Factura vencida
Saldo > 0
Sin arreglo

Resultado:

PV INACTIVO

Caso 3

Cliente
Factura vencida
Saldo > 0
Arreglo ACTIVO

Resultado:

PV ACTIVO

Caso 4

PV tiene 5 clientes
4 están al día
1 tiene mora

Resultado:

PV INACTIVO

Caso 5

PV tiene 5 clientes
1 moroso
Se registra arreglo

Resultado:

PV ACTIVO

Caso 6

PV bloqueado manualmente
Cliente no tiene mora

Resultado:

PV permanece INACTIVO

Esto es precisamente por lo que propuse:

MOTIVO_ESTADO
ORIGEN_ESTADO

42. Qué haría yo primero

No empezaría todavía creando todos los jobs.

Haría el desarrollo en este orden:

FASE 1

↓

Confirmar estructura real de tablas

↓

FASE 2

↓

Construir consulta de mora

↓

FASE 3

↓

Crear historial

↓

FASE 4

↓

Package de bloqueo/desbloqueo

↓

FASE 5

↓

Pruebas manuales

↓

FASE 6

↓
DBMS_SCHEDULER
↓
FASE 7
↓
Correo
↓
FASE 8
↓
Excel/CSV
↓
FASE 9
↓
Oracle Forms
↓
FASE 10
↓
Pruebas integrales

No pondría el Scheduler antes de comprobar que la consulta de mora y el procedimiento de bloqueo funcionan correctamente ejecutándolos manualmente.

Lo que necesito de ti para darte el código 100% adaptado

La arquitectura anterior ya la podemos utilizar como diseño definitivo, pero para darte **el código completo listo para copiar en PL/SQL Developer**, necesito los nombres reales de las columnas.

No necesito datos reales de clientes. Puedes sustituirlos por datos ficticios.

Pásame el resultado de estas consultas:

```
DESC CANAL;  
DESC DIVISION;  
DESC TERRITORIO;  
DESC AGENCIA;  
DESC PUNTO_VENTA;  
DESC CLIENTE;  
DESC SERVICIO;  
DESC FACTURA;
```

Y, si existen:

```
DESC PAGO;  
DESC ARREGLO_PAGO;
```

También necesito saber **qué versión de Oracle Database tienes** (por ejemplo 11g, 12c, 18c, 19c, 21c) y, si la conoces, la versión de Forms Builder.

Con esos datos puedo construirte la **versión 2 completa**, ya no ficticia: CREATE TABLE del historial, VIEW VW_CLIENTES_MORA, **package completo**, procedimientos de bloqueo/desbloqueo, manejo de arreglo de pago, DBMS_SCHEDULER, configuración SMTP, generación del archivo adjunto, correo a

los 3 destinatarios, consultas de monitoreo de los jobs y el diseño del formulario Oracle Forms con triggers WHEN - BUTTON - PRESSED, PRE - QUERY, WHEN - VALIDATE - ITEM, etc.

Y una recomendación importante: **no me envíes contraseñas, credenciales SMTP ni datos reales de clientes**; para el desarrollo podemos trabajar completamente con datos ficticios y luego sustituir únicamente la configuración de infraestructura.